Lab 10

6TiSCH and RPL: Channels, DODAG and Trickle

Communication Technologies for the Internet of Things | Lecture 10

Aim and context

Lecture 10 presents TSCH, with its channel hopping over slotframes, and RPL, with its root-oriented graph, objective function and Trickle Timer. This lab reproduces those mechanisms with a Python program, so that the numbers from the lecture can be computed, changed and compared.

The work needs no radios, no hardware and no virtual machine, and runs on any computer with Python 3. It is meant to be done during the class, at each student's pace, and is **not an assignment to hand in**.

> **Note**
>
> The program is a teaching model. It computes the frequency of a TSCH cell with the formula from the standard, chooses parents with shortest-path algorithms and simulates a simplified version of the Trickle Timer. It does not replace a network simulator.

Learning outcomes

- Compute the actual channel of a TSCH cell over successive slotframe iterations.
- Build a DODAG with a hop-count objective function and an ETX one, and compare the parent choices.
- Predict how the topology reacts to a link failure.
- Relate the Trickle Timer interval to the volume of signalling.

Requirements

- A computer running Windows, macOS or Linux, with Python 3.9 or later.
- Estimated duration: 40 to 60 minutes.

How to use this guide

Each step presents, in this order, the aim, the context, the theoretical background, the commands to run and the expected result. Read the whole step before running the first command. The boxes always have the same meaning:

Command to run

```
Command to run in the terminal.
```

Expected result (values may vary)

```
Result that should be displayed.
```

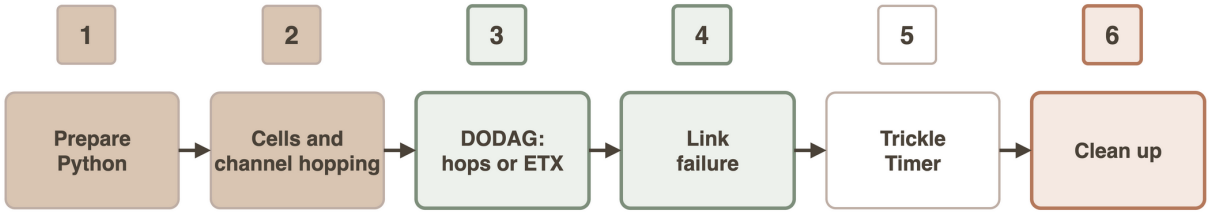
Analyse

A moment to pause and interpret the result, on your own or in discussion with colleagues.

Possible error and fix

An error that may occur and how to fix it.

Overview



A single Python program with no external libraries: each command shows a mechanism from the lecture

How to read the figure

The six steps read from left to right. The first prepares the program and the second computes the frequencies of a TSCH cell. Steps 3 and 4, in green, build the DODAG and observe the reaction to a failure. Step 5 shows the Trickle Timer, and step 6 closes the work.

Step 1 — Preparing the program

Aim. Save the lab program and confirm that Python runs it.

Theoretical background

The program uses only the Python standard library. Each program command matches a step of the guide: `tsch`, `dodag`, `fail` and `trickle`. With no argument, it runs all four.

Create the file `rpl_lab.py` with the contents below. The file is also available in the `code` folder of the lecture material.

Contents of file `rpl_lab.py`

```
"""6TiSCH and RPL lab (Lecture 10): channel hopping, DODAG parent selection and
    Trickle timer.

Run:  python3 rpl_lab.py [tsch|dodag|fail|trickle]
No external libraries are needed.
"""
import heapq
import sys

# ---------------------------------------------------------------- TSCH
HOPPING = [16, 17, 23, 18, 26, 15, 25, 22, 19, 11, 12, 13, 24, 14, 20, 21]  # RFC
    8180 default
SLOTFRAME = 101


def channel(asn, channel_offset):
    """Actual IEEE 802.15.4 channel of a cell at a given Absolute Slot Number."""
    return HOPPING[(asn + channel_offset) % len(HOPPING)]


def tsch():
    slot, ch_off = 3, 1
    print(f"cell slotOffset={slot} channelOffset={ch_off}, slotframe of {SLOTFRAME
    } slots")
    for it in range(8):
        asn = it * SLOTFRAME + slot
        print(f"  iteration {it}: ASN={asn:4d} -> channel {channel(asn, ch_off)}")


# ---------------------------------------------------------------- DODAG
# link -> ETX (expected transmissions); the graph is undirected here for
    simplicity
```

```
LINKS = {
    ("R", "A"): 1.0, ("R", "B"): 2.4, ("R", "C"): 4.0,
    ("A", "C"): 1.8, ("B", "C"): 1.1, ("A", "D"): 1.1,
    ("C", "E"): 1.2, ("D", "E"): 2.0, ("D", "F"): 1.0, ("E", "F"): 1.3,
}


def neighbours(links):
    g = {}
    for (a, b), w in links.items():
        g.setdefault(a, {})[b] = w
        g.setdefault(b, {})[a] = w
    return g


def dodag(links, metric):
    """Dijkstra from the root R. metric 'hops' uses 1 per link, 'etx' uses the
    link ETX."""
    g = neighbours(links)
    cost = {"R": 0.0}
    parent = {}
    heap = [(0.0, "R")]
    while heap:
        c, u = heapq.heappop(heap)
        if c > cost.get(u, 1e9):
            continue
        for v, etx in g[u].items():
            nc = c + (1.0 if metric == "hops" else etx)
            if nc < cost.get(v, 1e9) - 1e-9:
                cost[v], parent[v] = nc, u
                heapq.heappush(heap, (nc, v))
    return parent, cost


def show(links, metric):
    parent, cost = dodag(links, metric)
    for n in sorted(parent):
        print(f"  {n}: parent {parent[n]}, rank/cost {cost[n]:.1f}")
    return parent


def dodag_cmd():
    print("Objective function by hop count:")
    p1 = show(LINKS, "hops")
    print("Objective function by ETX:")
    p2 = show(LINKS, "etx")
    diff = [n for n in sorted(p1) if p1[n] != p2[n]]
    print("nodes that choose a different parent:", ", ".join(diff) or "none")
```

```
def fail():
    print("Before the failure (ETX):")
    before = show(LINKS, "etx")
    broken = dict(LINKS)
    del broken[("A", "D")]
    print("After the failure of link A-D (ETX):")
    after = show(broken, "etx")
    print("parents that changed:", ", ".join(f"{n}: {before[n]}->{after[n]}" for n
     in sorted(before) if before[n] != after[n]) or "none")


# ------------------------------------------------------------------ Trickle
def trickle(total=300, imin=1.0, doublings=6, reset_at=None):
    """Instants at which a DIO is sent (simplified Trickle: one DIO at the middle
    of each interval)."""
    t, interval, sends = 0.0, imin, []
    imax = imin * (2 ** doublings)
    while t < total:
        end, mid = t + interval, t + interval / 2.0
        if reset_at is not None and t <= reset_at < end:
            if mid <= reset_at:
                sends.append(round(mid, 1))
            t, interval, reset_at = reset_at, imin, None   # inconsistency: back
    to Imin
            continue
        sends.append(round(mid, 1))
        t, interval = end, min(interval * 2, imax)
    return sends


def trickle_cmd():
    stable = trickle()
    print(f"stable network: {len(stable)} DIOs in 300 s; intervals grow to {1.0 *
    2 ** 6:.0f} s")
    print("first sends:", stable[:8])
    reset = trickle(reset_at=100)
    print(f"inconsistency at t=100 s: {len(reset)} DIOs in 300 s")
    after = [s for s in reset if s > 100][:6]
    print("sends right after the reset:", after)


CMDS = {"tsch": tsch, "dodag": dodag_cmd, "fail": fail, "trickle": trickle_cmd}

if __name__ == "__main__":
    which = sys.argv[1] if len(sys.argv) > 1 else None
    for name, fn in CMDS.items():
```

```
        if which in (None, name):
            print(f"== {name}")
            fn()
```

Command to run

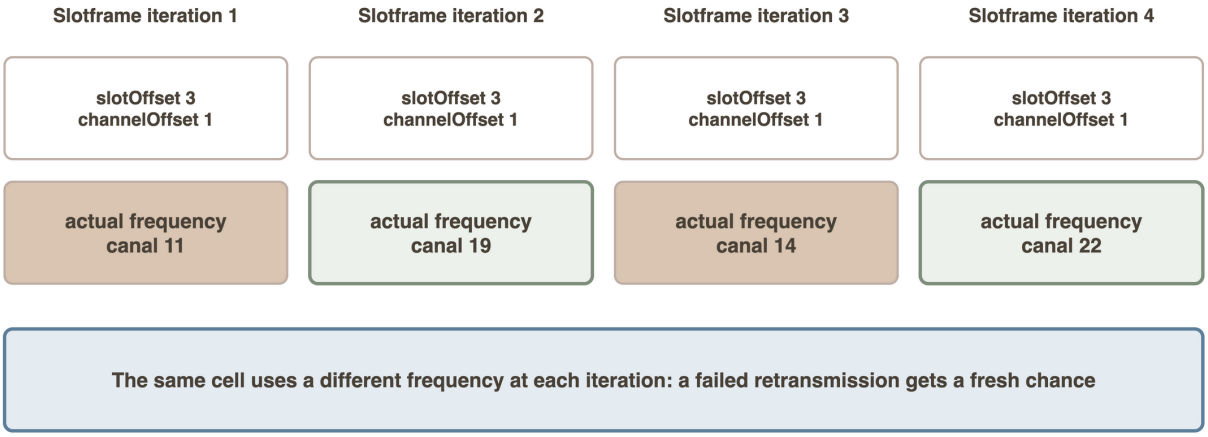
```
python3 rpl_lab.py tsch
```

Possible error and fix

python3: command not found on Windows: use `python rpl_lab.py tsch`.

Step 2 — TSCH cells and channel hopping

Aim. Compute the actual channel of a cell over eight slotframe iterations.



How to read the figure

Each column is a slotframe iteration. The upper box gives the cell, which is always the same, and the lower one the actual frequency, which changes at each iteration.

Theoretical background

The frequency of a cell is given by $F[(\text{ASN}+\text{channelOffset}) \bmod n]$, where F is the hopping sequence, n its length and ASN the absolute slot number. Since ASN grows at each iteration, the same cell lands on a different frequency. The sequence used is the RFC 8180 default, with 16 channels.

To analyse in this step: the relation between ASN, channel offset and frequency.

Expected result (values may vary)

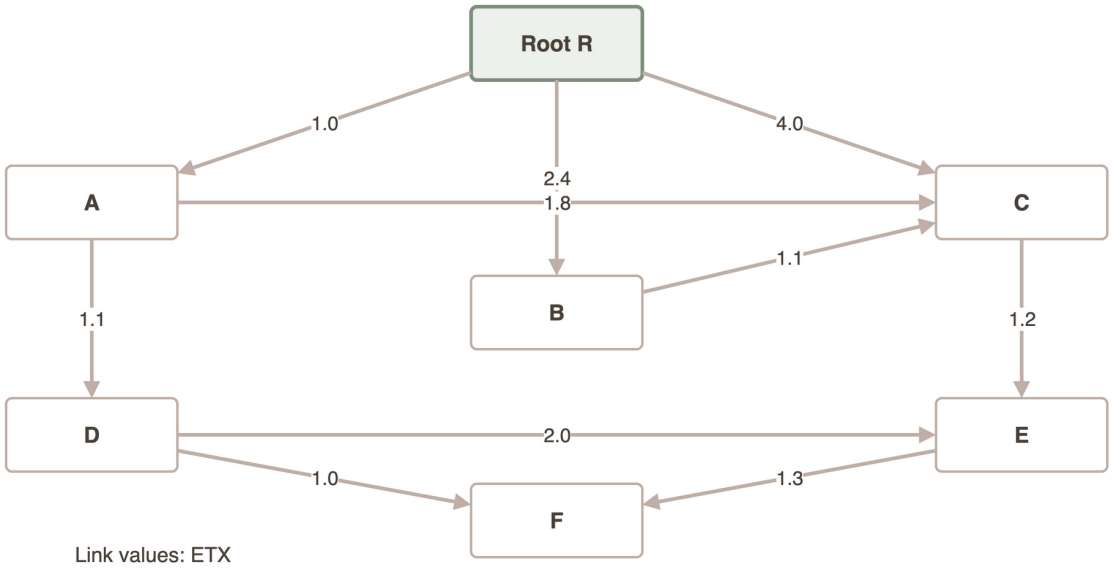
```
cell slotOffset=3 channelOffset=1, slotframe of 101 slots
  iteration 0: ASN=   3 -> channel 26
  iteration 1: ASN= 104 -> channel 11
  iteration 2: ASN= 205 -> channel 20
  iteration 3: ASN= 306 -> channel 18
  iteration 4: ASN= 407 -> channel 19
  iteration 5: ASN= 508 -> channel 14
  iteration 6: ASN= 609 -> channel 23
  iteration 7: ASN= 710 -> channel 22
```

Analyse

Change `ch_off` to 2 and predict the eight channels before running. Can two cells with the same slot and different channel offsets coincide on the same frequency in an iteration? Why?

Step 3 — The DODAG: hops or ETX

Aim. Build the graph with two objective functions and compare the parent chosen by each node.



How to read the figure

Node R is the root. Each link has its ETX, which estimates the number of transmissions needed to deliver successfully. The direct link between R and C looks short, but has an ETX of 4.0, whereas the path through A costs 2.8.

Theoretical background

The hop-count objective function adds 1 per link, and the ETX function adds the ETX of each link. A node's rank is the cost of the best path to the root, and the preferred parent is the neighbour through which that path passes. The program uses Dijkstra's algorithm, which gives the minimum-cost path, the same result RPL obtains in a distributed way.

To analyse in this step: which nodes change parent and why.

Command to run

```
python3 rpl_lab.py dodag
```

Expected result (values may vary)

```
Objective function by hop count:
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 1.0
  C: parent R, rank/cost 1.0
  D: parent A, rank/cost 2.0
  E: parent C, rank/cost 2.0
  F: parent D, rank/cost 3.0
Objective function by ETX:
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 2.4
  C: parent A, rank/cost 2.8
  D: parent A, rank/cost 2.1
  E: parent C, rank/cost 4.0
  F: parent D, rank/cost 3.1
nodes that choose a different parent: C
```

Analyse

Node C chooses different parents in the two functions. What is the real cost, in expected transmissions, of the path chosen by the hop-count function? And node E, which depends on C, does it have the same parent? What happens to its rank?

Step 4 — A link failure

Aim. Remove a link and observe how the topology reorganises.

Theoretical background

When a link stops working, nodes that used it as their best-cost path recompute their rank and may switch parent. This is the repair behaviour the DODAG offers without operator intervention. The cost is, as a rule, a larger rank and a longer path for the affected nodes.

Command to run

```
python3 rpl_lab.py fail
```

Expected result (values may vary)

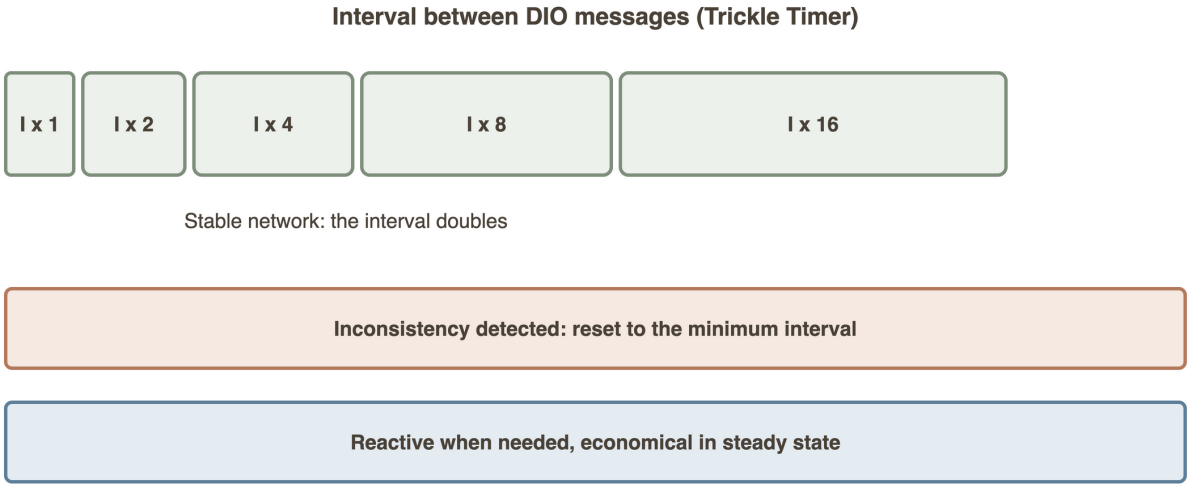
```
Before the failure (ETX):
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 2.4
  C: parent A, rank/cost 2.8
  D: parent A, rank/cost 2.1
  E: parent C, rank/cost 4.0
  F: parent D, rank/cost 3.1
After the failure of link A-D (ETX):
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 2.4
  C: parent A, rank/cost 2.8
  D: parent E, rank/cost 6.0
  E: parent C, rank/cost 4.0
  F: parent E, rank/cost 5.3
parents that changed: D: A->E, F: D->E
```

Analyse

Which nodes changed parent and which kept it despite a worse rank? Change the program to remove the C–E link and predict the result.

Step 5 — The Trickle Timer

Aim. Observe how the interval between DIOs grows in a stable network and is reset when an inconsistency is detected.



How to read the figure

The green blocks represent ever longer intervals, which double while the network stays stable. The red box says what happens when an inconsistency is detected.

Theoretical background

The Trickle Timer doubles the interval between emissions up to a maximum, while neighbours announce consistent information. On detecting an inconsistency, the interval goes back to the minimum, and the new information spreads within seconds. The program model emits one DIO in the middle of each interval, which is a simplification of the real mechanism, where the instant is random and redundant messages are suppressed.

To analyse in this step: the number of messages emitted in 300 seconds with and without an inconsistency.

Command to run

```
python3 rpl_lab.py trickle
```

Expected result (values may vary)

```
stable network: 10 DIOs in 300 s; intervals grow to 64 s
first sends: [0.5, 2.0, 5.0, 11.0, 23.0, 47.0, 95.0, 159.0]
inconsistency at t=100 s: 16 DIOs in 300 s
sends right after the reset: [100.5, 102.0, 105.0, 111.0, 123.0, 147.0]
```

Analyse

How many extra DIOs did the inconsistency cost? How would this number vary if Imin were 4 s? What is the advantage, on a battery-powered device, of a stable network emitting almost no signalling?

Step 6 — Cleaning up

Nothing was installed: just delete the file `rpl_lab.py`.

Review questions

1. Why does the same TSCH cell land on different frequencies in successive iterations?
2. In what situation does the hop-count objective function choose a worse path than the ETX one?
3. Which nodes have to recompute their rank when a link fails?
4. What changes in energy consumption when the Trickle Timer reaches its maximum interval?

Summary

- **Channel hopping** makes a cell use different frequencies at each iteration, which stabilises links.
- The **DODAG** chooses parents by the objective function criterion, and the ETX function protects against bad links.
- **Repair** is automatic: affected nodes recompute their rank and switch parent.
- The **Trickle Timer** reduces signalling in steady state and reacts quickly to change.